

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №1
по дисциплине Построение и анализ алгоритмов

Студент гр. 4343

Роменский М.А.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

Задание

Вар. 5р. Рекурсивный бэктрекинг.

Базовая часть

У Вовы много квадратных обрезков доски. Их стороны (размер) изменяются от 1 до $N-1$, и у него есть неограниченное число обрезков любого размера. Но ему очень хочется получить большую столешницу — квадрат размера $N \times N$. Он может получить её, собрав из уже имеющихся обрезков (квадратов). Внутри столешницы не должно быть пустот, обрезки не должны выходить за пределы столешницы и не должны перекрываться. Кроме того, Вова хочет использовать минимально возможное число обрезков.

Входные данные: размер столешницы — одно целое число N ($2 \leq N \leq 20$).

Выходные данные: одно число K — минимальное количество обрезков (квадратов), из которых можно построить столешницу заданного размера N . Далее должны идти K строк, каждая из которых содержит три целых числа x , y и w , задающие координаты левого верхнего угла ($1 \leq x, y \leq N$) и длину стороны соответствующего обрезка (квадрата).

Индивидуализация

Возможность задать список квадратов (от 0 до N^2 квадратов в списке), которые **обязательно должны быть использованы** в покрытии квадрата со стороной N .

Входные данные

- N — размер столешницы.
- M — количество обязательных квадратов ($0 \leq M \leq N^2$).
- Далее M строк: x , y , w — координаты левого верхнего угла и сторона обязательного квадрата.

Выходные данные: либо K и K строк с квадратами, либо No solution, если замощения с обязательными квадратами не существует.

Описание алгоритма

Общий случай

Для решения задачи используется метод **рекурсивного перебора с возвратом** (backtracking). Алгоритм последовательно размещает квадраты на доске, начиная от максимально возможного размера ($N-1$) к минимальному (1), стартуя с левой верхней клетки.

На каждом шаге рекурсии:

1. Ищется первая свободная клетка (сверху-вниз, слева-направо).
2. Определяется максимально возможный размер квадрата, который можно поставить в эту клетку.
3. Перебираются все возможные размеры от максимального к 1.
4. Если квадрат можно поставить — он ставится, и рекурсия продолжается.
5. После возврата из рекурсии квадрат убирается (откат).

Для варианта 5p добавляется учёт обязательных квадратов:

- При постановке квадрата проверяется, не перекрывает ли он ещё не использованный обязательный квадрат (если не совпадает с ним).
- Ведётся массив `required_used`, отмечающий использованные обязательные квадраты.
- При полном заполнении поля проверяется, что **все** обязательные квадраты использованы.

Способ хранения частичных решений

Для хранения текущего состояния используется двумерный массив `board` размером $N \times N$, где:

- `board[i][j] = 0` — свободная клетка,
- `board[i][j] > 0` — занятая клетка.

Для хранения координат квадратов в текущем частичном решении используется вектор `current`, содержащий пары $((x, y), w)$, где x, y — координаты левого верхнего угла, w — сторона квадрата.

Для хранения лучшего найденного решения используется вектор `best_solution`.

Для учёта обязательных квадратов используются:

- вектор `required` — список обязательных квадратов $((x, y), w)$;
- вектор `required_used` — булев массив, отмечающий использованные обязательные квадраты.

Рекурсивная функция

Сигнатура:

```
void backtrack(int count, int depth)
```

Назначение: рекурсивный поиск минимального числа квадратов для полного покрытия поля с учётом обязательных квадратов.

Аргументы:

- `count` — текущее количество использованных квадратов;
- `depth` — глубина рекурсии (для отладочного вывода).

Возвращаемое значение: отсутствует, так как функция модифицирует глобальные переменные `best_count`, `best_solution`, `board`, `current`, `required_used`.

Логика работы

1. **Отсечение по рекорду:** если `count >= best_count` и поле не заполнено — выход.
2. **Проверка заполненности:** если `empty_area == 0`, проверяется, все ли обязательные квадраты использованы. Если да — обновляется рекорд.
3. **Поиск первой свободной клетки:** если свободных клеток нет — выход.

4. **Определение максимального размера квадрата:** $\text{max_side} = \min(N - x, N - y, N - 1)$.
5. **Нижняя оценка:** $\text{lb} = \text{ceil}(\text{empty_area} / \text{max_side}^2)$. Если $\text{count} + \text{lb} \geq \text{best_count}$ — отсечение.
6. **Перебор размеров:** от max_side до 1:
 - проверка `can_place`;
 - проверка `conflicts_with_required` (не перекрывает ли обязательный);
 - постановка квадрата, добавление в `current`, отметка обязательного;
 - рекурсивный вызов `backtrack(count + 1, depth + 1)`;
 - откат: снятие квадрата, удаление из `current`, снятие отметки.

Оценка сложности:

Временная сложность:

В худшем случае без оптимизаций алгоритм перебирает все возможные комбинации квадратов. На каждом шаге выбирается первая свободная клетка, и для неё перебираются возможные размеры квадратов. Число возможных последовательностей квадратов экспоненциально.

С оптимизациями (отсечения по рекорду, нижняя оценка, перебор от большего к меньшему) сложность значительно лучше, но в худшем случае остаётся экспоненциальной: $O(c^S)$, где $c < 2$, $S = N^2$

В лучшем случае (поле полностью покрывается несколькими большими квадратами) сложность близка к линейной: $O(S)$, где $S = N^2$.

Сложность по памяти:

- Хранение доски: $O(N^2)$.
- Стек рекурсии: $O(K)$, где K — количество квадратов в решении.

- Общая память: $O(N^2) + O(K) = O(N^2)$.

Так как хранение доски доминирует над стеком рекурсии, общая память определяется размером доски.

Оптимизации алгоритма:

1. **Отсечение по лучшему решению (верхняя граница):** если текущее количество квадратов больше или равно лучшему найденному, ветка отсекается.
2. **Приоритет больших квадратов:** перебор размеров от максимального к минимальному позволяет быстрее находить хорошие решения.
3. **Нижняя оценка:** оценка минимально необходимого количества квадратов для оставшихся клеток ($lb = \text{ceil}(\text{empty_area} / \text{max_side}^2)$).
4. **Отсечение по обязательным квадратам:** если ставимый квадрат перекрывает ещё не использованный обязательный (и не совпадает с ним) — ветка отсекается.
5. **Проверка заполненности через empty_area:** вместо обхода всей доски используется целое число — оставшаяся площадь.
6. **Чётный N — быстрый путь:** если N чётное и нет обязательных квадратов, ответ сразу 4 (четыре квадрата $N/2 \times N/2$).

Тестирование:

Входные данные	Выходные данные	Комментарий
2	4; 1 1 1; 1 2 1; 2 1 1; 2 2 1	Минимальный размер: четыре квадрата 1x1
3	6; 1 1 2; 2 1 1; 3 1 1; 1 2 1; 2 2 1; 3 2 1; 2 3 1; 3 3 1	Нечётный N: 2x2 + пять 1x1
4	4; 1 1 2; 1 3 2; 3 1 2; 3 3 2	Чётный N: четыре квадрата 2x2
5	8; 1 1 3; 1 4 2; 3 4 2; 1 5 2; 3 5 2; 5 1 2; 4 1 2; 5 3 2	Нечётный N, простое число
6	4; 1 1 3; 1 4 3; 4 1 3; 4 4 3	Чётный N: четыре квадрата 3x3
7	9; 1 1 4; 1 5 3; 4 5 2; 4 7 1; 5 1 3; 5 4 1; 5 7 1; 6 4 2; 6 6 2	Пример из задания
20	4; 1 1 10; 1 11 10; 11 1 10; 11 11 10	Максимальный размер, чётный N
7 0	9; 1 1 4; 1 5 3; 4 5 2; 4 7 1; 5 1 3; 5 4 1; 5 7 1; 6 4 2; 6 6 2	Базовая часть без обязательных (M=0)
7 1; 4 4 4	9; 4 4 4; 1 1 4; 1 5 3; 4 5 2; 4 7 1; 5 1 3; 5 4 1; 5 7 1; 6 4 2; 6 6 2	Обязательный квадрат 4x4 в (4,4)
7 1; 1 1 7	No solution	Сторона 7 вне диапазона 1..6
7 2; 1 1 3; 2 2 3	No solution	Обязательные квадраты пересекаются
4 4; 1 1 2; 1 3 2; 3 1 2; 3 3 2	4; 1 1 2; 1 3 2; 3 1 2; 3 3 2	Все обязательные совпадают с оптимумом
8 0	4; 1 1 4; 1 5 4; 5 1 4; 5 5 4	Чётный N без обязательных: 4 квадрата 4x4
9 0	6; 1 1 6; 1 7 3; 4 7 3; 7 1 3; 7 4 3; 7 7 3	Нечётный N, делится на 3
11 0	11; 1 1 6; 1 7 5; 6 7 5; 1 12 1; 12 1 1; 12 2 1;	Простое N, сложный случай

	12 3 1; 12 4 1; 12 5 1; 12 6 1; 12 7 1	
13 0	11; 1 1 7; 1 8 6; 7 8 6; 1 14 1; 14 1 1; 14 2 1; 14 3 1; 14 4 1; 14 5 1; 14 6 1; 14 7 1	Простое N
16 0	4; 1 1 8; 1 9 8; 9 1 8; 9 9 8	Чётный N: четыре квадрата 8x8

Вывод

В ходе лабораторной работы был реализован рекурсивный алгоритм с возвратом для задачи покрытия квадратного поля минимальным числом квадратов. Для варианта 5p добавлена возможность задавать список обязательных квадратов, которые должны быть использованы в покрытии.

Алгоритм успешно находит оптимальное решение для N до 20 и корректно обрабатывает обязательные квадраты, включая проверку их корректности и отсечение веток, конфликтующих с ними. Применённые оптимизации позволяют значительно ускорить поиск.